# Práctica: Creación de un sitio web multipágina con Flask y Bootstrap

## 1. Objetivo

Desarrollar una aplicación web con Flask que contenga varias páginas HTML, utilizando rutas, plantillas Jinja2, paso de variables desde Python y herencia de plantillas. Se deberá modificar la estructura de navegación y la organización general de las páginas mediante los conocimientos básicos de Bootstrap revisados en clase.

## 2. Descripción de la actividad

A partir del proyecto de ejemplo trabajado en clase, desarrolla un sitio web para una **biblioteca universitaria**. El sitio permitirá presentar información general de la biblioteca, su catálogo, sus servicios y los medios de contacto.

Se deberá conservar la estructura general de la aplicación Flask, pero modificar los nombres de las páginas, su contenido y el diseño del menú de navegación.

Todas las páginas deberán compartir una plantilla base llamada `layout.html`, de la cual heredarán la estructura HTML y el menú de navegación. Cada página tendrá su propio contenido y recibirá desde Flask las variables necesarias para mostrar su título y un mensaje introductorio.

## 3. Páginas que se deberán desarrollar

Todas las páginas recibirán las variables título y mensaje que deberá ser utilizado como título de la página y contenido.

### Página 1. Inicio (`index.html`)

Presentará información general de la biblioteca universitaria.

Contenido mínimo:

- Nombre de la biblioteca.
- Mensaje de bienvenida.
- Breve descripción de su propósito.
- Horario general de atención.

### Página 2. Catálogo (`catalogo.html`)

Presentará información general sobre los materiales disponibles en la biblioteca.

Contenido mínimo:

- Título de la página.
- Mensaje introductorio.
- Lista de categorías de materiales: libros, revistas, tesis y recursos digitales.
- Una breve descripción de cada categoría.

### Página 3. Servicios (`servicios.html`)

Describirá los principales servicios que ofrece la biblioteca.

Contenido mínimo:

- Título de la página.
- Mensaje introductorio.
- Lista de servicios, como préstamo de libros, consulta en sala, acceso a recursos digitales y orientación bibliográfica.
- Una breve descripción de cada servicio.

### Página 4. Contacto (`contacto.html`)

Presentará los medios de contacto de la biblioteca.

Contenido mínimo:

- Título de la página.
- Mensaje introductorio.
- Dirección de la biblioteca.
- Correo electrónico institucional ficticio.
- Horario de atención.

Los datos utilizados deberán ser ficticios o corresponder a información autorizada para el ejercicio.

## 4. Modificaciones solicitadas

### A. Modificar las rutas de Flask

En el archivo `server.py`, define las rutas necesarias para las cuatro páginas del sitio. Utiliza `render_template()` para mostrar cada plantilla y enviar, al menos, las variables `titulo` y `mensaje`.

Las rutas deberán corresponder a las páginas de Inicio, Catálogo, Servicios y Contacto.

### B. Crear una nueva plantilla base

Modifica `layout.html` para adaptarlo al sitio de la biblioteca. La plantilla deberá contener la estructura HTML general, la referencia a Bootstrap, el título del documento, el menú de navegación y el bloque donde se insertará el contenido de cada página.

### C. Cambiar el tipo de navegación

Sustituye el menú del proyecto original por una navegación horizontal de Bootstrap con las siguientes características:

- El nombre de la biblioteca aparecerá como identificador principal del sitio.
- Los enlaces de navegación serán Inicio, Catálogo, Servicios y Contacto.
- El menú deberá utilizar un estilo visual diferente al del ejemplo original, modificando las clases de Bootstrap correspondientes.
- Los enlaces deberán utilizar `url_for()` para dirigirse a las rutas de Flask.
- La navegación deberá definirse únicamente en `layout.html`, de modo que aparezca automáticamente en todas las páginas.

No es necesario incorporar menús desplegables ni otros componentes avanzados.

### D. Modificar la presentación de las páginas

Utiliza `container-fluid` para organizar el contenido de las páginas, tal como se hizo en el proyecto de ejemplo.

No es necesario utilizar ningun otro componente que todavía no se hayan estudiado.

## 5. Requisitos técnicos

1. Utilizar Flask y el archivo `server.py` como punto de entrada.
2. Crear cuatro rutas para las páginas solicitadas.
3. Utilizar `render_template()` para cargar las plantillas.
4. Enviar las variables `titulo` y `mensaje` desde las funciones de las rutas.
5. Crear la plantilla base `layout.html`.
6. Utilizar `{% extends "layout.html" %}` en las cuatro plantillas hijas.
7. Definir el contenido particular de cada página mediante `{% block content %}`.
8. Implementar el menú de navegación en la plantilla base.
9. Utilizar `url_for()` para generar los enlaces de navegación.
10. Utilizar `container-fluid` para organizar el contenido de las páginas.
11. Verificar que todas las páginas funcionen y que el menú permita navegar entre ellas.

## 6. Estructura sugerida del proyecto

```
biblioteca/
├── server.py
├── templates/
│   ├── layout.html
│   ├── index.html
│   ├── catalogo.html
│   ├── servicios.html
│   └── contacto.html
└── requirements.txt
```

## 7. Entregables

Agregar un archivo README.md con una explicación de cada elemento implementado en el ejercicio, utiliza código Markdown para resaltar títtulos, subtítulos y viñetas o cualquier otro elemento de la explicación.

- URL de github.

## 8. Criterios de evaluación

Se evaluará el funcionamiento de las rutas, el uso de variables, la correcta implementación de la herencia de plantillas, la modificación del menú de Bootstrap, la organización del contenido y el cumplimiento de los requisitos solicitados.